

What Is Software Design: 13 Years Later

By Jack W. Reeves

People have occasionally asked whether I did any follow-on writing to my “What Is Software Design” article. The answer has basically been “No, not really.” I want to make it clear that that is not because I forgot about it or otherwise changed my mind. Allow me to offer a bit of explanation.

When the article appeared, I hoped—actually expected—that I would get some type of rebuttal from some sort of industry “expert.” I was looking forward to this since part of my reason for writing the article had been hopes of stimulating discussion within the software industry about the overall software development process. Nothing happened.

There were no letters to the editor that I know about and nothing ever sent directly to me. *C++ Journal* became defunct shortly after that issue, and I figured my article had gone to that great land fill in the sky that swallows most publications. I went on to doing other things. It wasn't until 1997 or 1998 that I got an email from Bob Martin (who had just taken over as editor of the *C++ Report*) letting me know there was a wiki page about my article on Ward Cunningham's *c2.com* web site. This was—quite literally—the first time I knew anybody had read my article (other than the people I personally gave a copy to).

I started to follow the discussions on the wiki page and occasionally on some news groups, but deliberately stayed out of them myself for several reasons: a) I was focused on certain other things at the time, b) it was pretty obvious that other people who had accepted what I was trying to say were just as qualified—maybe more so—to argue the points as I would have been (I specifically remember Michael Feathers writing), and c) last but not least, it still looked to me like there was a lot of opposition to the concept. Unfortunately, most of the arguments sounded pretty much like the ones I had been dealing with for almost 15 years by that point (remember, I had had the idea almost 10 years before I wrote the article).

I had grown tired of trying to deal with people who were totally incapable of getting past their own pre-conceptions to even consider the idea rationally. It was like trying to explain that the French speak a different language to someone who is convinced that “different language” really just means “different dialect of English”. No matter what you say, they will parse your arguments against their beliefs and either dismiss you out of hand, or patronize you with their counter arguments. I had seen a number of projects where “design it in the code” worked, but even the people on such projects often refused to accept the reality. My level of cynicism about being able to improve things was very high.

It still is, but I think it is time I made some attempt to actually defend myself, rather than let other people do it. Therefore, what I am going to do is address some of the most common criticisms I have seen about “What Is Software Design?”.

A. Initially, the most common criticism I would see can be summarized as “If source code is the design, then programmers are designers; but obviously they are not, therefore source code cannot be the design.” Nobody states it that baldly, but when you parse what they do say, it comes down to the same thing. These are circular arguments that start with the assumption that programming/coding is a manufacturing type of activity. In logic, this is known as a “Begging the Question” fallacy. In essence, these people say “your assumption (i.e. source code is the design) contradicts my assumption (i.e. programmers are assembly workers), therefore your assumption must be wrong.”

Someone might suggest that I am doing the same thing, i.e. starting with the assumption that source code is a design. I accept that—up to a point. While I will admit that a lot of the article reads like an attempt to prove that “source code is the design”, that was not really what I was trying to do. The following quote is from the beginning of the article:

“This article assumes that final source code is the real software design and then examines some of the consequences of that assumption. I may not be able to prove that this point of view is correct, but I hope to show that it does explain some of the observed facts of the software industry, ...”

I did not set out to prove that “source code is the design”; I will readily concede that what is a “design” is to some extent a matter of definition. The point of the article was to try to show how this assumption led to much better explanations of numerous observed *facts*. I am still waiting for anyone to offer better explanations based upon alternative assumptions.

B. These days, thanks in part to the rise of Extreme Programming and other Agile Methods, people are starting to accept (grudgingly) that programmers are not assembly line drones. Unfortunately, that doesn’t mean they are willing to accept the concept of “the source code is the design”. The arguments can be summarized by the example that is still on the wiki page:

“As for throwing the whole design thing out, and just designing in code...
Hahahahahahahahah no really Hahahahahahahahah :)”

This really makes me angry. For reasons that I do not understand, reasonably intelligent people insist upon confusing the concept of design as *process* versus design as *product*. You would think that anyone who passed high school would understand the difference between the process of writing a paper (for example) and the paper itself. Certainly, you would expect anyone with a college background to understand that there are often lots of different ways to arrive at the same solution.

Nevertheless, people keep insisting that my contention of “the source code is the design” means “don’t do design, just code.” I never said anything of the sort. What I did say was:

“In software engineering, we desperately need good design at all levels. In particular, we need good top level design. The better the early design, the easier detailed design will be. Designers should use anything that helps. Structure charts, Booch diagrams, state tables, PDL, etc.—if it helps, then use it.”

Today, I would phrase it differently. I would say we need good architectures (top level design), good abstractions (class design), and good implementations (low level design). I would also say something about using UML diagrams or CRC cards to explore alternatives. Nevertheless, I will not back away from the following statement:

“We must keep in mind, however, that these tools and notations are not a software design. Eventually, we have to create the real software design, and it will be in some programming language. Therefore, we should not be afraid to code our designs as we derive them.”

This is fundamental. I am not arguing that we should not “do design.” However you want to approach the *process*, I simply insist that you have not completed the process until you have written *and* tested the code.

Personally, I think a person with his feet on the desk staring at the ceiling can be “doing design” just as seriously as someone playing with UML diagrams in ROSE. I have always known that you are better off if you put some real thought into what you are trying to do before actually doing it. People differ widely in what helps them think, however. Some people use pencil and paper. Others like white boards or even computer tools. Some people like to bounce ideas off of other people, others like peace and quiet. Some people feel comfortable with diagrams like UML. Others prefer CRC cards.

What approach they choose doesn't matter; until someone starts insisting that these intermediate designs should be products in their own right. It's the code that matters. If you get good code, does it really matter how it came about? If you don't get good code, does it really matter how much other garbage you made people do before they wrote the bad code?

Everybody that has been in this business any length of time has seen plenty of examples where someone obviously sat down and coded the first thing that popped into their mind. Later, when it became obvious that there were shortcomings with the approach, there was too much blood, sweat, and "skin" invested in the code to scrap it and do something better. Fine, we all know a little thought can go a long way.

On the other hand, any of us who has spent time on a *traditional* development project with its strict rules forbidding the writing of a single line of code until the "design" is completed and reviewed and approved, etc. knows you can waste a hell of a lot of time producing documents that are out of date literally days after the actual coding starts. Why bother?

You think we could find some happy medium of "enough" design effort, but not too much. There is no such thing. The only way we validate a software design is by building it and testing it. There is no silver bullet, and no "right way" to do design. Sometimes an hour, a day, or even a week spent thinking about a problem can make a big difference when the coding actually starts. Other times, 5 minutes of testing will reveal something you never would have thought about no matter how long you tried. We do the best we can under the circumstances, and then refine it.

One last comment: I also did not say that the only necessary documentation is the source code. I specifically pointed out in the article:

"...auxiliary documentation is as important for a software project as it is for a hardware project."

The source code may be the master design document, but it is seldom the only one necessary.

B'. I cannot resist making a remark about a side issue that often comes up in discussions about Extreme Programming and Agile methods that is related to the above. This is often phrased as a question: what about the Less Able Programmer? The issue seems to be that only the very best programmers can "design" and "code" at the same time. To offset this, we must have all those intermediate design steps and products mentioned above to make up for the lack of experience and talent of the average programmer.

To me, this is like asking “what do we do about the less able physician?” I know the practice of medicine and software development are not analogous, but bear with me for a moment. An awful lot of the practice of medicine is pretty much rote (we joke about “take two aspirin and call me tomorrow”). Nevertheless, the medical profession still insists upon some pretty high standards of intelligence, education, and experience before someone is allowed to call themselves an MD. In other words, we want our doctors to know what they are doing.

In software development, questions about the less able programmer really come down to trying to substitute a process for intelligence, aptitude, and experience. Apparently, a lot of people think that if we force people to create enough UML diagrams (or whatever), have enough reviews, and otherwise follow a detailed process, that eventually they will figure out what they are doing and code it correctly. There is no evidence that such approaches have worked in the past, and I see no reason to believe they will work in the future. In fact, my own experience says that properly using tools such as UML involves a considerable level of expertise and experience in its own right.

C. Another argument I have seen questions the contention that the goal of an engineering effort is some type of documentation. Some people argue that the goal of engineering is a “product” and that real engineers often “build” things and those “things” are as much a product of engineering as any documentation.

This argument tries to sidestep the question of “What is a Software Design?” by implying a parallel between the “things” that other engineers build and what software developers create. Frankly, this is nonsense. I will concede that there are engineers who build “things” with little or no formal design documentation, and I suspect that even in those cases there is probably some design documentation (even if it is on the back of an envelope). In any case, I think we can safely say that such projects produce only one-off products and are usually done by individuals.

When an “engineering” effort starts involving more than a couple of people, or when it has a formal manufacturing phase, then documentation starts to loom larger and larger as the actual product of the engineering effort. You better believe that the engineers at Toyota or Motorola produce documentation, and we’ll not even think about the engineers at Boeing or Lockheed. So, while it might be true that a lot of engineers do things besides producing design documents, anyone who calls himself an engineer knows what a design document in his field looks like, and probably produces such more often than not. Can we say the same for “software engineers”?

Incidentally, this contention regarding engineers and documentation was not mine originally, but instead something I picked up from an article in *Datamation* back in 1979. I agree with it completely however.

D. One final but fairly minor argument I have seen is that source code is still too high level to be a design. At least one critic wanted to call source code the “specification.” His (or her) take was that the real design is what comes out of the compiler. In some sense this is just a matter of definition, but I still disagree with it.

The generally accepted definition is that a “specification” states the *what*, which is followed by a design document that details the *how*. While there is a certain amount of flexibility allowed of the compiler in determining the *how* of object code, there is certainly no creativity involved. And that is where I draw the line. When the document is detailed enough, complete enough, and unambiguous enough that it can be interpreted mechanistically, whether by a computer or by an assembly line worker, then you have a design document. If it still requires creative human interpretation, then you don’t.

In software development, the design document is a source code listing.

###

Copyright ©2005 by Jack W. Reeves.

<continued next page>